

Kütüphane Yönetim Sistemi

Backend API — Test Dokümanı

Bu doküman projenin tüm işlevlerinin uçtan uca test edilmesini adım adım içerir. Her JSON gövdesi ve terminal komutu ayrı bir kod kutusundadır; doğrudan kopyalayıp yapıştırabilirsiniz. Testler Swagger arayüzü (localhost:3000/api) üzerinden yürütülür.

Teknoloji	NestJS + TypeORM + PostgreSQL
Altyapı	Docker Compose (backend + postgres konteynerleri)
Kimlik doğrulama	JWT + bcrypt, USER / ADMIN rolleri
Test kullanıcısı	kullanici@test.com / 123456 (USER + ADMIN)

Hazırlık — Ortamın Başlatılması

Docker Desktop açık olmalı. Aşağıdaki komutlar proje klasöründe çalıştırılır. Terminal komutları için Docker'ın çalıştığı pencereden AYRI bir terminal (cmd) kullanılır.

Projeyi başlat (temiz kurulum için --build):

```
docker compose up --build
```

Temiz veritabanıyla başlamak için (tabloları sıfırlar, roller yeniden oluşur):

```
docker compose down -v  
docker compose up --build
```

Swagger arayüzü:

```
http://localhost:3000/api
```

ADIM 1 — Kimlik Doğrulama ve Yetkilendirme

Amaç: Sisteme kimin girebileceğini ve kimin ne yapabileceğini doğrulamak.

TEST 1.1 Kayıt olma (register)

POST /auth/register — Swagger'da Try it out → Body:

Body

```
{
  "email": "kullanici@test.com",
  "password": "123456",
  "fullName": "Normal Kullanıcı"
}
```

Beklenen: 201 Created. id, email, fullName döner; passwordHash dönmez.

TEST 1.2 Geçersiz veriyle kayıt (validation)

POST /auth/register — hatalı veri:

Body

```
{
  "email": "gecersiz",
  "password": "12",
  "fullName": ""
}
```

Beklenen: 400 Bad Request. Türkçe hata mesajları (geçerli e-posta, en az 6 karakter, ad soyad zorunlu).

TEST 1.3 Giriş yapma (login)

POST /auth/login — doğru bilgilerle:

Body

```
{
  "email": "kullanici@test.com",
  "password": "123456"
}
```

Beklenen: 201. accessToken (JWT) döner. Bu token Authorize butonuna girilir.

TEST 1.4 Yanlış şifreyle giriş

POST /auth/login — yanlış şifre:

Body

```
{
  "email": "kullanici@test.com",
  "password": "yanlissifre"
}
```

Beklenen: 401 Unauthorized ('E-posta veya şifre hatalı' — bilinçli olarak belirsiz mesaj).

TEST 1.5 Token ile korumalı endpoint

GET /auth/me — Authorize'a token girildikten sonra çağrılır.

Beklenen: Kullanıcı bilgisi döner: userId, email, roles. Token olmadan 401.

TEST 1.6 Yetki kontrolü (USER, ADMIN endpoint'ine)

GET /auth/admin-test — sadece USER rolündeki token ile.

Beklenen: 403 Forbidden. RolesGuard USER'ı reddeder — yetkilendirmenin çalıştığının kanıtı.

Kullanıcıyı ADMIN yapma (katalog testleri için gerekli)

Ayrı terminalde (cmd) çalıştır, sonra yeniden login ol (roller token içinde saklandığı için yeni token gerekir):

```
docker exec -it kutuphane-db psql -U kutuphane -d kutuphanedb -c "INSERT INTO user_roles (\"userId\", \"rolesId\") VALUES ((SELECT id FROM users WHERE email='kullanici@test.com'), (SELECT id FROM roles WHERE name='ADMIN'));"
```

Doğrulama: yeniden login → yeni token ile GET /auth/admin-test artık 200 döner.

ADIM 2 — Katalog Yönetimi ve Arama

Amaç: İçerik yönetimi ve arama / filtreleme / sayfalama / sıralama. Ekleme işlemleri ADMIN token'ı ile.

TEST 2.1 Yazar, kategori, yayınevi ekleme

ADMIN token'ı ile, tam bu sırayla ekle (id'ler tahmin edilebilir olsun diye). Her biri ayrı POST isteğidir:

POST /authors (id 1 olmalı)

```
{ "name": "J.K. Rowling" }
```

POST /authors (id 2)

```
{ "name": "George Orwell" }
```

POST /categories (id 1)

```
{ "name": "Fantastik" }
```

POST /categories (id 2)

```
{ "name": "Distopya" }
```

POST /publishers (id 1)

```
{ "name": "Bloomsbury" }
```

Beklenen: Her istek 201 Created döner.

TEST 2.2 Yetki testi (token olmadan ekleme)

Authorize'dan Logout → POST /authors çağrılır:

Body

```
{ "name": "Test" }
```

Beklenen: 401 Unauthorized. Ekleme korunuyor. Sonra tekrar Authorize olunur.

TEST 2.3 Kitap oluşturma (çoklu ilişki, doğru id'lerle)

POST /books — iki kitap, tek tek gönderilir:

Kitap 1 — POST /books

```
{
  "title": "Harry Potter",
  "isbn": "978-0001",
  "publishedYear": 1997,
  "totalCopies": 2,
  "publisherId": 1,
  "authorIds": [1],
  "categoryIds": [1]
}
```

Kitap 2 — POST /books

```
{
  "title": "1984",
  "isbn": "978-0002",
  "publishedYear": 1949,
  "totalCopies": 1,
  "publisherId": 1,
  "authorIds": [2],
  "categoryIds": [2]
}
```

Beklenen: 201. Yanıtta yazar + kategori + yayınevi ilişkileri dolu döner.

TEST 2.4 Arama (search)

Tarayıcıda veya Swagger'da:

URL

```
http://localhost:3000/books?search=harry
```

Beklenen: Sadece 'Harry Potter' döner (total: 1), yazarı J.K. Rowling.

TEST 2.5 Kategoriye göre filtreleme

Distopya kategorisi:

URL

```
http://localhost:3000/books?category=Distopya
```

Beklenen: Sadece '1984' döner (yazar George Orwell).

TEST 2.6 Yazara göre filtreleme

Rowling'in kitapları:

URL

```
http://localhost:3000/books?author=Rowling
```

Beklenen: Sadece 'Harry Potter' döner.

TEST 2.7 Sayfalama (pagination)

Sayfa başına 1 kitap:

Sayfa 1

```
http://localhost:3000/books?limit=1&page=1
```

Sayfa 2

```
http://localhost:3000/books?limit=1&page=2
```

Beklenen: Her sayfada 1 kitap; total: 2, totalPages: 2.

TEST 2.8 Sıralama (sorting)

Başlığa göre alfabetik:

URL

```
http://localhost:3000/books?sortBy=title
```

Beklenen: '1984' önce, 'Harry Potter' sonra. Hocanın istediği alfabetik listeleme.

ADIM 3 — İş Kuralları (Projenin Kalbi)

Amaç: Ödünç/iade, stok, gecikme cezası, rezervasyon, yorum ve istatistik kurallarını doğrulamak.

TEST 3.1 Ödünç alma ve stok düşmesi

POST /borrowings (Harry Potter, 2 kopya):

Body

```
{ "bookId": 1 }
```

Beklenen: 201. remainingCopies: 1, dueDate 5 gün sonrası. Doğrulama: ?search=harry ile availableCopies 2→1.

TEST 3.2 Aynı kitabı tekrar alma (engel)

POST /borrowings — yine aynı kitap:

Body

```
{ "bookId": 1 }
```

Beklenen: 400 'Bu kitabı zaten ödünç aldınız'.

TEST 3.3 İkinci kitabı alma

POST /borrowings (1984):

Body

```
{ "bookId": 2 }
```

Beklenen: 201. Artık 2 aktif ödünç var (maksimum).

TEST 3.4 Üçüncü kitap — max 2 kuralı

Önce 3. bir kitap eklenir (farklı isbn), sonra POST /borrowings:

Body

```
{ "bookId": 3 }
```

Beklenen: 400 'En fazla 2 kitap ödünç alabilirsiniz'.

TEST 3.5 Normal iade (gecikme yok)

GET /borrowings/my ile borrowing id öğrenilir, sonra:

PATCH /borrowings/1/return (URL'deki id kendi kaydına göre)

Beklenen: 200 'Kitap iade edildi', lateDays: 0, penalty: 'Ceza yok'. Stok geri artar (availableCopies 2).

TEST 3.6 Gecikme cezası (simülasyon)

Ayrı terminalde dueDate geçmişe çekilir, sonra PATCH /borrowings/2/return çağrılır:

Terminal (cmd)

```
docker exec -it kutuphane-db psql -U kutuphane -d kutuphanedb -c "UPDATE borrowings SET \"dueDate\" = NOW() - INTERVAL '4 days' WHERE id=2;"
```

Beklenen: 200, lateDays > 0, penalty = 'X gün ödünç alma yasağı' (geciken gün × 3). Math.ceil nedeniyle gün 1 fazla yuvarlanabilir — normaldir.

TEST 3.7 Cezalıyken ödünç alma (engel)

Ceza alındıktan sonra POST /borrowings:

Body

```
{ "bookId": 1 }
```

Beklenen: 400 'Cezalısınız. ... tarihine kadar ödünç alamazsınız'. Ceza döngüsü kapanır.

TEST 3.8 Rezervasyon (stok bitince)

Önce ceza sıfırlanır ve stok 0 yapılır (iki komut ayrı ayrı), sonra POST /reservations:

Terminal 1 — cezayı sıfırla

```
docker exec -it kutuphane-db psql -U kutuphane -d kutuphanedb -c "UPDATE users SET \"bannedUntil\"=NULL WHERE email='kullanici@test.com';"
```

Terminal 2 — stoğu 0 yap

```
docker exec -it kutuphane-db psql -U kutuphane -d kutuphanedb -c "UPDATE books SET \"availableCopies\"=0 WHERE id=1;"
```

Body

```
{ "bookId": 1 }
```

Beklenen: 201, queuePosition: 1. Rezervasyon yalnızca stok yokken anlamlıdır.

TEST 3.9 Aynı kitaba tekrar rezervasyon (engel)

POST /reservations — yine aynı kitap:

Body

```
{ "bookId": 1 }
```

Beklenen: 409 'Bu kitap için zaten rezervasyonunuz var'. @Unique(['user','book']) kısıtı.

TEST 3.10

Stok varken rezervasyon (engel)

POST /reservations (1984, stok var):

Body

```
{ "bookId": 2 }
```

Beklenen: 400 'Kitap şu an mevcut, doğrudan ödünç alabilirsiniz'.

TEST 3.11

Admin istatistik

ADMIN token'ı ile iki GET isteği:

URL 1

```
http://localhost:3000/borrowings/stats/popular-books
```

URL 2

```
http://localhost:3000/borrowings/stats/active-users
```

Beklenen: En çok ödünç alınan kitaplar ve en aktif kullanıcılar borrowCount ile döner (GROUP BY + COUNT). USER ile 403.

TEST 3.12

Yorum ekleme (review)

Yalnızca ödünç alınan kitaba yorum yapılabilir. POST /reviews:

Body

```
{ "bookId": 1, "rating": 5, "comment": "Harika bir kitap" }
```

Beklenen: 201 'Yorum eklendi'.

TEST 3.13

Aynı kitaba tekrar yorum (engel)

POST /reviews — yine aynı kitap:

Body

```
{ "bookId": 1, "rating": 3 }
```

Beklenen: 409 'Bu kitaba zaten yorum yaptınız'. Kullanıcı başına tek yorum.

TEST 3.14

Geçersiz puan

POST /reviews — puan aralık dışında:

Body

```
{ "bookId": 2, "rating": 7 }
```

Beklenen: 400 'Puan en fazla 5 olmalıdır'. Puan 1-5 kuralı.

TEST 3.15

Yorumları listeleme (passwordHash kontrolü)

GET /reviews/book/1:

URL

```
http://localhost:3000/reviews/book/1
```

Beklenen: Yorum kullanıcı bilgisiyle döner; passwordHash yanıtta olmaz (@Exclude + interceptor).

Otomatik Testler (Jest)

Unit test servisi izole ederek mantığı, e2e testi gerçek HTTP isteğiyle tüm zinciri doğrular.

Unit test (AuthService — bilgisayarda çalışır):

```
npm run test auth.service
```

Integration / e2e test (konteyner içinde çalıştırılır):

```
docker exec -it kutuphane-api npm run test:e2e
```

Not: e2e testi doğrudan bilgisayardan çalıştırılırsa DB_HOST=postgres adresine bağlanamaz (bu isim sadece Docker ağı içinde geçerlidir); bu yüzden konteyner içinden çalıştırılır.